

RELATORIO DA ATIVIDADE 3

Compressao DCT e Descritores de Imagem

Processamento Digital de Imagens

Aluno: Yago Melo Da Costa

Tema: Moda

Data: 07/06/2026

Sumario

1. Objetivo	4
2. Imagens Utilizadas	5
3. Metodologia	6
4. Questao 1 - Compressao DCT (estilo JPEG)	7
5. Questao 2 - Descritores de Imagem	10
6. Conclusao	13
7. Codigos Implementados	14
8. Referencias	20

1. Objetivo

Este trabalho tem como objetivo explorar e consolidar conceitos de compressão, representação e descrição de imagens digitais, por meio da implementação prática de técnicas fundamentais. A Questão 1 implementa uma versão simplificada de compressão de imagens baseada na Transformada Discreta do Cosseno (DCT), inspirada no padrão JPEG, analisando as perdas introduzidas no processo. A Questão 2 desenvolve um conjunto de descritores estatísticos e estruturais para caracterizar imagens em tons de cinza, comparando diferentes padrões visuais.

2. Imagens Utilizadas

As imagens utilizadas estão relacionadas ao tema do trabalho final. A Figura abaixo apresenta as duas imagens em tons de cinza utilizadas nos experimentos: imagem1 foi usada na Questão 1 (compressão DCT) e ambas foram utilizadas na Questão 2 (descritores).



Imagem 1 (imagem1.jpeg)



Imagem 2 (imagem2.jpeg)

3. Metodologia

Todas as transformações foram implementadas manualmente em Python, sem uso de funções prontas das bibliotecas. O carregamento e salvamento de imagens utilizou apenas PIL/Pillow. Os cálculos de DCT e IDCT foram feitos com loops explicitamente baseados na definição matemática da transformada. Os descritores foram calculados percorrendo os pixels da imagem via operações NumPy básicas. As bibliotecas matplotlib e fpdf foram usadas apenas para geração de gráficos e do relatório, respectivamente.

4. Questao 1 - Compressao DCT (estilo JPEG)

A Questao 1 implementa o pipeline de compressao JPEG simplificado: (1) conversao para tons de cinza, (2) divisao em blocos 8x8, (3) aplicacao da DCT 2D manual em cada bloco, (4) quantizacao com a matriz padrao JPEG, (5) dequantizacao e (6) aplicacao da IDCT 2D para reconstrucao. Foram testados cinco valores do fator de escala de quantizacao (q_scale): 0.5, 1.0, 2.0, 4.0 e 8.0. Valores menores preservam mais informacao (qualidade maior), valores maiores aumentam a compressao e a perda de qualidade.

4.1 Imagem Original (tons de cinza)



Imagem original convertida para tons de cinza

4.2 Imagens Reconstruidas por q_scale



q=0.5 (Alta qualidade)



q=1.0 (Padrao JPEG)



q=2.0

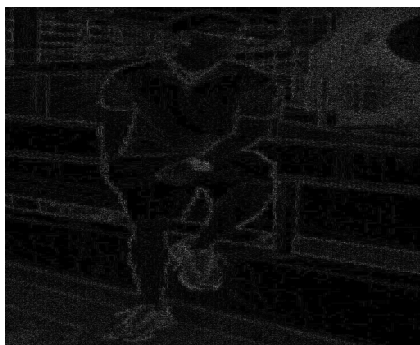


q=4.0

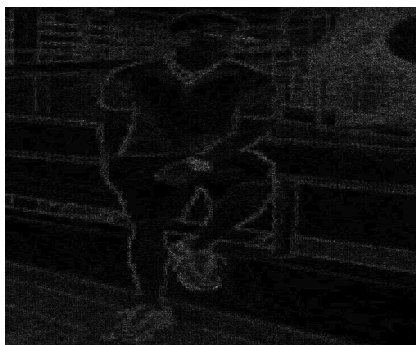


q=8.0 (Alta compressao)

4.3 Imagens de Diferenca (|original - reconstruida|)



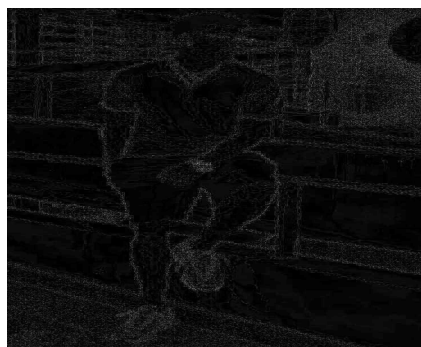
Diferenca q=0.5 (Alta qualidade)



Diferenca q=1.0 (Padrao JPEG)



Diferenca q=2.0



Diferença q=4.0



Diferença q=8.0 (Alta compressao)

4.4 Metricas de Qualidade

A tabela abaixo apresenta o PSNR (Peak Signal-to-Noise Ratio) e o MAE (Mean Absolute Error) para cada configuracao de q_scale. PSNR acima de 40 dB indica qualidade excelente; abaixo de 30 dB, perda perceptivel significativa.

q_scale	PSNR (dB)	MAE
0.5	47.2461	0.7153
1.0	38.5843	1.9270
2.0	35.3761	2.8772
4.0	32.0876	4.2916
8.0	28.7637	6.5177

4.5 Analise dos Resultados

Com q_scale=0.5 a quantizacao e mais fina, preservando mais coeficientes DCT e resultando no maior PSNR. A medida que q_scale aumenta, a matriz de quantizacao amplifica os passos de arredondamento, descartando mais informacao de alta frequencia. Isso se manifesta visualmente como artefatos de bloco (blocky artifacts) e perda de bordas e texturas finas. As imagens de diferenca evidenciam que as maiores discrepancias concentram-se nas regioes de alto detalhe, confirmando o comportamento teorico da DCT: coeficientes de alta frequencia sao os primeiros a ser eliminados. O artefato de bloco torna-se nitidamente visivel para q_scale $\geq$ 4.0.

5. Questao 2 - Descritores de Imagem

A Questao 2 implementa um conjunto de descritores para caracterizar imagens em tons de cinza, combinando medidas estatísticas globais e informacoes estruturais. Os descritores calculados sao: media, variancia, desvio padrao, energia, entropia de Shannon e variacao espacial (diferenca absoluta entre pixels vizinhos horizontal e vertical).

5.1 Comparacao Visual das Imagens



Media: 135.1
Var: 5475.7
Energia: 23733.4
Entropia: 7.48 bits
Var.Espacial: 4.39



Media: 132.5
Var: 6514.2
Energia: 24072.4
Entropia: 7.51 bits
Var.Espacial: 11.95

Comparacao visual com descritores

5.2 Imagens Individuais em Tons de Cinza

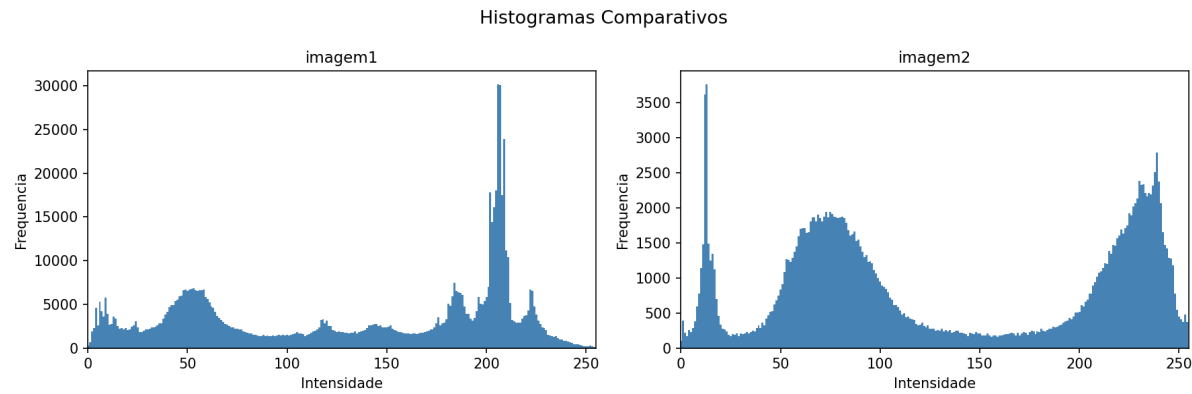


Imagem imagem1



Imagem imagem2

5.3 Histogramas Comparativos



Histogramas de intensidade das imagens

5.4 Grafico Radar dos Descritores

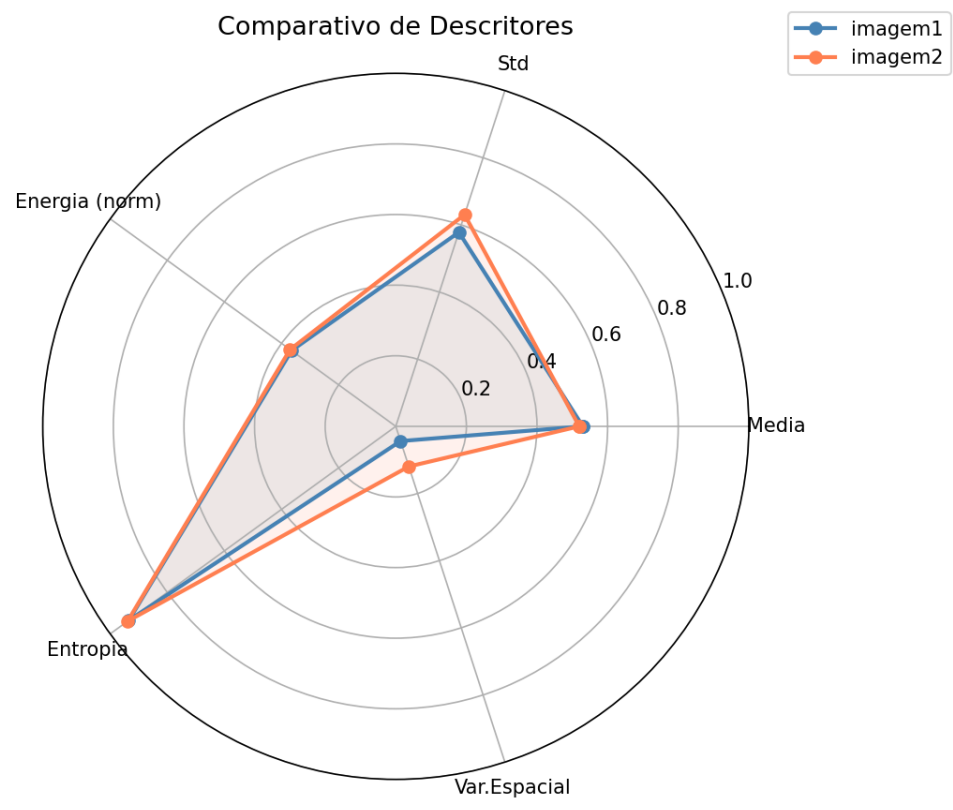


Grafico radar comparativo dos descritores normalizados

5.5 Tabela de Descritores

Descritor	imagem1	imagem2
Media	135.12	132.51

Variancia	5475.68	6514.17
Desvio Padrao	74.00	80.71
Energia	23733.44	24072.42
Entropia (bits)	7.48	7.51
Var.Esp. Horiz.	4.12	12.48
Var.Esp. Vert.	4.65	11.42
Var.Esp. Total	4.39	11.95

5.6 Interpretacao dos Resultados

Os descritores revelam diferencas significativas entre as imagens. Imagens com alta variacao espacial total possuem grande quantidade de detalhes e texturas complexas, enquanto regioes com baixa variacao correspondem a areas homogeneas (como ceu ou fundos uniformes). A entropia reflete a diversidade do histograma: imagens com distribuicao de intensidade mais uniforme apresentam maior entropia. A energia e inversamente relacionada a uniformidade: imagens homogeneas tendem a concentrar os pixels em poucos niveis, aumentando a energia. Esses descritores sao uteis em sistemas de recuperacao de imagens, classificacao de texturas e pre-processamento para algoritmos de aprendizado de maquina, servindo como vetor de caracteristicas compacto para diferenciar padroes visuais distintos.

6. Conclusao

Os experimentos demonstraram como a DCT permite representar imagens de forma compacta por meio da concentracao de energia nos coeficientes de baixa frequencia. A quantizacao introduz perdas controlaveis pelo parametro q_scale : quanto maior o fator, maior a compressao e mais intensos os artefatos de bloco. Os descritores estatisticos e estruturais mostraram-se eficazes para diferenciar imagens com diferentes caracteristicas visuais, sendo a variacao espacial o indicador mais sensivel ao nivel de detalhe e textura presentes nas imagens. Ambas as tecnicas fornecem bases para aplicacoes praticas em compressao, classificacao e recuperacao de imagens digitais.

7. Codigos Implementados

Os codigos-fonte utilizados para gerar os resultados sao apresentados a seguir. O primeiro arquivo implementa a compressao DCT (Questao 1) e o segundo implementa os descritores de imagem (Questao 2).

questao1_dct.py

```
import os
import json
import math
import numpy as np
from PIL import Image

BLOCK_SIZE = 8

QUANT_MATRIX = np.array([
    [16, 11, 10, 16, 24, 40, 51, 61],
    [12, 12, 14, 19, 26, 58, 60, 55],
    [14, 13, 16, 24, 40, 57, 69, 56],
    [14, 17, 22, 29, 51, 87, 80, 62],
    [18, 22, 37, 56, 68, 109, 103, 77],
    [24, 35, 55, 64, 81, 104, 113, 92],
    [49, 64, 78, 87, 103, 121, 120, 101],
    [72, 92, 95, 98, 112, 100, 103, 99],
], dtype=np.float32)

def ensure_dir(path):
    os.makedirs(path, exist_ok=True)

def load_gray(path):
    return np.array(Image.open(path).convert('L'), dtype=np.float32)

def save_u8(arr, path):
    Image.fromarray(np.clip(arr, 0, 255).astype(np.uint8)).save(path)

def dct2d(block):
    N = BLOCK_SIZE
    result = np.zeros((N, N), dtype=np.float64)
    for u in range(N):
        for v in range(N):
            cu = 1.0 / math.sqrt(2) if u == 0 else 1.0
            cv = 1.0 / math.sqrt(2) if v == 0 else 1.0
            s = 0.0
            for x in range(N):
                for y in range(N):
                    s += (block[x, y] *
                        math.cos((2*x + 1)*u*math.pi / (2*N)) *
                        math.cos((2*y + 1)*v*math.pi / (2*N)))
            result[u, v] = (2.0 / N) * cu * cv * s
    return result

def idct2d(block):
    N = BLOCK_SIZE
    result = np.zeros((N, N), dtype=np.float64)
    for x in range(N):
```

```

    for y in range(N):
        s = 0.0
        for u in range(N):
            for v in range(N):
                cu = 1.0 / math.sqrt(2) if u == 0 else 1.0
                cv = 1.0 / math.sqrt(2) if v == 0 else 1.0
                s += (cu * cv * block[u, v] *
                      math.cos((2*x + 1)*u*math.pi / (2*N)) *
                      math.cos((2*y + 1)*v*math.pi / (2*N)))
            result[x, y] = (2.0 / N) * s
    return result

def quantize_block(dct_block, q_scale=1.0):
    scaled_q = QUANT_MATRIX * q_scale
    return np.round(dct_block / scaled_q)

def dequantize_block(quant_block, q_scale=1.0):
    scaled_q = QUANT_MATRIX * q_scale
    return quant_block * scaled_q

def process_image(gray, q_scale=1.0):
    h, w = gray.shape
    bh = (h // BLOCK_SIZE) * BLOCK_SIZE
    bw = (w // BLOCK_SIZE) * BLOCK_SIZE
    img = gray[:bh, :bw]
    reconstructed = np.zeros_like(img, dtype=np.float64)
    for i in range(0, bh, BLOCK_SIZE):
        for j in range(0, bw, BLOCK_SIZE):
            block = img[i:i+BLOCK_SIZE, j:j+BLOCK_SIZE].astype(np.float64) - 128.0
            dct = dct2d(block)
            quant = quantize_block(dct, q_scale)
            dequant = dequantize_block(quant, q_scale)
            idct = idct2d(dequant)
            reconstructed[i:i+BLOCK_SIZE, j:j+BLOCK_SIZE] = idct + 128.0
    return img, np.clip(reconstructed, 0, 255)

def psnr(original, reconstructed):
    mse = np.mean((original.astype(np.float64) - reconstructed.astype(np.float64)) ** 2)
    if mse == 0:
        return float('inf')
    return 10.0 * math.log10(255.0**2 / mse)

def mean_absolute_error(original, reconstructed):
    return float(np.mean(np.abs(original.astype(np.float64) - reconstructed.astype(np.float64))))

def main():
    import argparse
    p = argparse.ArgumentParser()
    p.add_argument('--input', required=True)
    p.add_argument('--outdir', default='Atividade3/resultados_q1')
    args = p.parse_args()

    ensure_dir(args.outdir)

    gray = load_gray(args.input)
    save_u8(gray, os.path.join(args.outdir, 'original_gray.png'))

    q_scales = [0.5, 1.0, 2.0, 4.0, 8.0]

```

```

results = {}

for qs in q_scales:
    original_crop, reconstructed = process_image(gray, q_scale=qs)
    tag = str(qs).replace('.', '_')
    out_path = os.path.join(args.outdir, f'reconstruida_q{tag}.png')
    save_u8(reconstructed, out_path)

    diff = np.abs(original_crop.astype(np.float64) - reconstructed)
    diff_norm = diff / diff.max() * 255.0 if diff.max() > 0 else diff
    diff_path = os.path.join(args.outdir, f'diferenca_q{tag}.png')
    save_u8(diff_norm, diff_path)

    psnr_val = psnr(original_crop, reconstructed)
    mae_val = mean_absolute_error(original_crop, reconstructed)

    results[tag] = {
        'q_scale': qs,
        'psnr': round(psnr_val, 4),
        'mae': round(mae_val, 4),
        'reconstructed': out_path,
        'diff': diff_path
    }
    print(f'q_scale={qs} | PSNR={psnr_val:.2f} dB | MAE={mae_val:.2f}')

json_path = os.path.join(args.outdir, 'resultados_q1.json')
with open(json_path, 'w') as f:
    json.dump(results, f, indent=2)

print(f'\nResultados salvos em {args.outdir}')

if __name__ == '__main__':
    main()

```

questao2_descritores.py

```

import os
import json
import math
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt

def ensure_dir(path):
    os.makedirs(path, exist_ok=True)

def load_gray(path):
    return np.array(Image.open(path).convert('L'), dtype=np.float32)

def save_u8(arr, path):
    Image.fromarray(np.clip(arr, 0, 255).astype(np.uint8)).save(path)

def compute_mean(img):
    return float(np.sum(img) / img.size)

def compute_variance(img, mean):
    return float(np.sum((img - mean) ** 2) / img.size)

```

```

def compute_energy(img):
    return float(np.sum(img.astype(np.float64) ** 2) / img.size)

def compute_spatial_variation(img):
    h, w = img.shape
    diff_h = np.abs(img[:, 1:].astype(np.float64) - img[:, :-1].astype(np.float64))
    diff_v = np.abs(img[1:, :].astype(np.float64) - img[:-1, :].astype(np.float64))
    mean_h = float(np.sum(diff_h) / diff_h.size)
    mean_v = float(np.sum(diff_v) / diff_v.size)
    total = float((np.sum(diff_h) + np.sum(diff_v)) / (diff_h.size + diff_v.size))
    return mean_h, mean_v, total

def compute_entropy(img):
    hist = np.zeros(256, dtype=np.float64)
    flat = img.astype(np.uint8).ravel()
    for val in flat:
        hist[val] += 1
    hist /= flat.size
    entropy = 0.0
    for p in hist:
        if p > 0:
            entropy -= p * math.log2(p)
    return float(entropy)

def compute_histogram(img):
    hist = np.zeros(256, dtype=np.int64)
    flat = img.astype(np.uint8).ravel()
    for val in flat:
        hist[val] += 1
    return hist

def describe_image(img):
    mean = compute_mean(img)
    variance = compute_variance(img, mean)
    std = math.sqrt(variance)
    energy = compute_energy(img)
    entropy = compute_entropy(img)
    var_h, var_v, var_total = compute_spatial_variation(img)
    return {
        'mean': round(mean, 4),
        'variance': round(variance, 4),
        'std': round(std, 4),
        'energy': round(energy, 4),
        'entropy': round(entropy, 4),
        'spatial_variation_horizontal': round(var_h, 4),
        'spatial_variation_vertical': round(var_v, 4),
        'spatial_variation_total': round(var_total, 4),
    }

def save_histogram_figure(hist_data, labels, outpath, title='Histogramas Comparativos'):
    fig, axes = plt.subplots(1, len(hist_data), figsize=(6 * len(hist_data), 4))
    if len(hist_data) == 1:
        axes = [axes]
    bins = np.arange(256)
    for ax, hist, label in zip(axes, hist_data, labels):
        ax.bar(bins, hist, width=1, color='steelblue', edgecolor='none')
        ax.set_title(label, fontsize=11)
        ax.set_xlabel('Intensidade')

```

```

        ax.set_ylabel('Frecuencia')
        ax.set_xlim(0, 255)
    plt.suptitle(title, fontsize=13)
    plt.tight_layout()
    plt.savefig(outpath, dpi=150)
    plt.close()

def save_comparison_figure(images, labels, descriptors, outpath):
    n = len(images)
    fig, axes = plt.subplots(1, n, figsize=(7 * n, 5))
    if n == 1:
        axes = [axes]
    for ax, img, label, desc in zip(axes, images, labels, descriptors):
        ax.imshow(img, cmap='gray', vmin=0, vmax=255)
        ax.set_title(label, fontsize=11)
        info = (
            f"Media: {desc['mean']:.1f}\n"
            f"Var: {desc['variance']:.1f}\n"
            f"Energia: {desc['energy']:.1f}\n"
            f"Entropia: {desc['entropy']:.2f} bits\n"
            f"Var.Espacial: {desc['spatial_variation_total']:.2f}"
        )
        ax.set_xlabel(info, fontsize=9)
        ax.set_xticks([])
        ax.set_yticks([])
    plt.tight_layout()
    plt.savefig(outpath, dpi=150)
    plt.close()

def save_radar_chart(desc_list, labels, outpath):
    categories = ['Media', 'Std', 'Energia (norm)', 'Entropia', 'Var.Espacial']
    N = len(categories)
    angles = [n / float(N) * 2 * math.pi for n in range(N)]
    angles += angles[:1]

    fig, ax = plt.subplots(figsize=(7, 7), subplot_kw=dict(polar=True))
    colors = ['steelblue', 'coral', 'green', 'purple', 'orange']

    for idx, (desc, label) in enumerate(zip(desc_list, labels)):
        values = [
            desc['mean'] / 255.0,
            desc['std'] / 128.0,
            min(desc['energy'] / 65025.0, 1.0),
            desc['entropy'] / 8.0,
            min(desc['spatial_variation_total'] / 100.0, 1.0)
        ]
        values += values[:1]
        ax.plot(angles, values, 'o-', linewidth=2,
              color=colors[idx % len(colors)], label=label)
        ax.fill(angles, values, alpha=0.1, color=colors[idx % len(colors)])

    ax.set_thetagrids(np.degrees(angles[:-1]), categories)
    ax.set_ylim(0, 1)
    ax.legend(loc='upper right', bbox_to_anchor=(1.3, 1.1))
    ax.set_title('Comparativo de Descriptores', fontsize=13, pad=20)
    plt.tight_layout()
    plt.savefig(outpath, dpi=150)
    plt.close()

def main():
    import argparse

```

```

p = argparse.ArgumentParser()
p.add_argument('--inputs', required=True, nargs='+')
p.add_argument('--labels', required=True, nargs='+')
p.add_argument('--outdir', default='Atividade3/resultados_q2')
args = p.parse_args()

ensure_dir(args.outdir)

images = []
labels = []
descriptors = []
histograms = []

for path, label in zip(args.inputs, args.labels):
    img = load_gray(path)
    save_u8(img, os.path.join(args.outdir, f'gray_{label}.png'))
    desc = describe_image(img)
    hist = compute_histogram(img)
    images.append(img)
    labels.append(label)
    descriptors.append(desc)
    histograms.append(hist)
    print(f'\n[{label}]')
    for k, v in desc.items():
        print(f'  {k}: {v}')

hist_path = os.path.join(args.outdir, 'histogramas_comparativos.png')
save_histogram_figure(histograms, labels, hist_path)

comparison_path = os.path.join(args.outdir, 'comparativo_imagens.png')
save_comparison_figure(images, labels, descriptors, comparison_path)

radar_path = os.path.join(args.outdir, 'radar_descritores.png')
save_radar_chart(descriptors, labels, radar_path)

results = {}
for label, desc in zip(labels, descriptors):
    results[label] = desc

json_path = os.path.join(args.outdir, 'resultados_q2.json')
with open(json_path, 'w') as f:
    json.dump(results, f, indent=2)

print(f'\nResultados salvos em {args.outdir}')

if __name__ == '__main__':
    main()

```

8. Referencias

GONZALEZ, R. C.; WOODS, R. E. Digital Image Processing. 4. ed. Pearson, 2018.

WALLACE, G. K. The JPEG still picture compression standard. IEEE Transactions on Consumer Electronics, 1992.

Documentacao NumPy. Disponivel em: <https://numpy.org/doc>

Documentacao Pillow. Disponivel em: <https://pillow.readthedocs.io>

Implementacoes proprias em Python.